SDI Investment Property Marketplace

System Documentation

What has been built, how to run it, and who can see what

Repository	`github.com/neilgreene/property-management`
Branch	`main`
Commits	13
Database	PostgreSQL 16 (row-level security; requires 15+)
Application	Node.js 18+ (no framework, no runtime dependencies beyond `pg`)
Automated checks	63 worker tests plus 4 SQL walkthroughs, all passing
Deployment status	Local development only. Not deployed, not internet-facing.

Every fact in this document was extracted from the repository and from a freshly built database, not written from memory. Regenerate it with `python3 docs/generate_system_documentation.py`.

This page intentionally left blank.

Contents

1. What This Is

A working system for a private investment property marketplace: vetted properties listed with financial analysis, browsable without revealing addresses, with the full detail unlocked only after an investor signs and pays the platform fee agreement. Agents see only their own assignments. Staff see everything.

1.1 The problem it solves first

The commercial model depends on one thing holding: an unpaid visitor must not be able to obtain a property's street address. Everything else is ordinary software; that is not. So the visibility rules are enforced **inside the database**, by row-level security policies and column grants, rather than by application code.

This matters because it changes what an attacker has to defeat. If the rule lives in application code, anyone who finds an unguarded query, an API bug, or a direct database connection gets the address. Here, the address is withheld by the database itself: a caller who writes their own SQL, bypasses the application entirely, and connects directly still cannot read it.

1.2 Three visibility bands

Band	Contains	Released to	Enforced by
1 — Public	City, price, cap rate, NOI, beds, baths, year built	Anyone, including anonymous visitors	RLS policy per role
2 — Gated	Street address, unit, parcel number, seller disclosure, true coordinates	Investors who signed AND paid; the assigned agent; staff	Masking in a `security_invoker` view, keyed on a data predicate
3 — Internal	Acquisition cost, source channel, staff notes, margin	Staff only	Column-level GRANT (a hard ACL)

Coordinates degrade rather than disappear. An ungated viewer gets a deterministic offset of roughly one kilometre, seeded on the property id, so a map still renders a neighbourhood and repeated page loads cannot be averaged to recover the true point.

2. How To Run It

2.1 Docker — nothing installed but Docker

```
git clone https://github.com/neilgreene/property-management
cd property-management
docker compose up
```

Builds the database from `sql/`, seeds it, and serves the demo at **http://localhost:3000**. PostgreSQL is exposed on `localhost:5432` (database `sdi`, user `postgres`, password `postgres`). `docker compose down -v` discards the database.

2.2 Local PostgreSQL 16+ and Node 18+

```
./run.sh
```

Loads all eleven schema files, runs all four SQL walkthroughs, runs the 63 worker tests, then starts the demo. It assumes `psql` and `createdb` work as your own user, which is the default for Postgres.app and Homebrew.

2.3 Individual pieces

Command	What it does
`psql -d sdi -f sql/05_tests.sql`	Security walkthrough: 11 checks, 5 of them attacks
`psql -d sdi -f sql/07_ghl_tests.sql`	GoHighLevel bridge: 7 checks
`psql -d sdi -f sql/10_review_tests.sql`	Review queue actions: 7 checks
`psql -d sdi -f sql/14_pipeline_tests.sql`	Deal visibility and stage history: 9 checks
`cd worker && npm test`	63 unit and end-to-end tests

2.4 The integration worker

Deliberately not started by a bare `docker compose up`, because it needs real GoHighLevel credentials and there is no point running it without them.

```
GHL_TOKEN=... GHL_LOCATION_ID=... docker compose --profile worker up
```

Variable	Where it comes from
`GHL_TOKEN`	GoHighLevel: Settings → Private Integrations. Scoped to an entire sub-account.
`GHL_LOCATION_ID`	The sub-account id, visible in the GoHighLevel URL
`GHL_WEBHOOK_PUBLIC_KEY`	GoHighLevel's published webhook verification key (PEM)

The token is read from the environment and is never written into any file in the repository. It grants access to the whole sub-account — contacts, invoices, transactions — so it must never reach browser-side code.

3. Users and Access

3.1 There is no login system yet

> **This must be stated plainly, because it is the most likely thing to be misread. The system has no authentication: no password for a person, no session, no sign-in page. The demo presents a persona switcher, and choosing a persona makes the web tier assume the matching database role and set that person's id for the duration of one transaction.**

That is deliberate for this stage and costs nothing later. The database contract is identical either way: in production the persona and actor id come out of an authenticated session instead of a dropdown, and not one policy, view or grant changes. Authentication is the missing piece — the authorisation model beneath it is complete and tested.

3.2 Seeded people

Created by `sql/04_seed.sql`. These are demonstration records, not real people; the addresses and financials attached to them are invented.

Name	Role	Email	Fee agreement	Brand	What they demonstrate
Jessica Pool	admin	`jpool@yahoo.com`	n/a	BRAND_A	Full staff access, all bands
Dan Beitor	admin	`dan@example.com`	n/a	BRAND_A	Full staff access, all bands
Tom Bradbury	agent	`tom@example.com`	n/a	BRAND_A	4 assigned properties, incl. an unpublished draft
Priya Raman	agent	`priya@example.com`	n/a	BRAND_A	A second agent, to prove agents are isolated
Ruth Okonkwo	investor	`ruth@example.com`	signed	BRAND_A	Gate open: sees addresses and true coordinates
Marcus Pell	investor	`marcus@example.com`	not signed	BRAND_A	Gate shut: same query, address withheld
Ines Duarte	investor	`ines@example.com`	signed	KAVADOO	The second brand, at concierge pricing

Ruth and Marcus are the pair that carries the argument: identical role, identical SQL, and the only difference between them is one timestamp in a data column. The address appears for one and not the other with no application logic involved at all.

3.3 Database roles

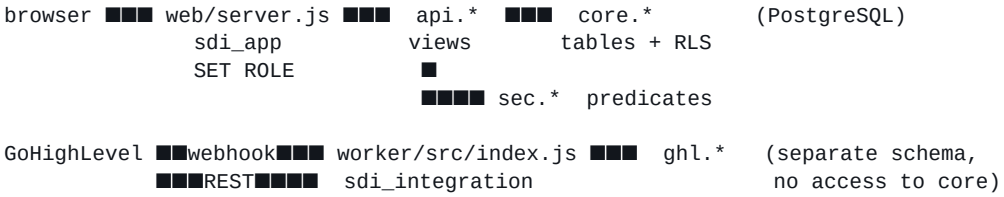
Application roles are created `NOLOGIN` and passwordless on purpose. `sdi_app` is `NOINHERIT`, so it holds no privileges of its own and must assume exactly one persona role per transaction — the application cannot forget to assume a role and accidentally run with more authority than it should.

Role	Purpose	Login	Demo password
`sdi_app`	The only role the web tier connects as	yes	`demo_app_pw`
`sdi_public`	Anonymous visitors. Band 1 only	no	assumed via SET ROLE

Role	Purpose	Login	Demo password
`sdi_investor`	Investors. Band 2 if the gate is open	no	assumed via SET ROLE
`sdi_agent`	Agents. Band 2 on assigned properties	no	assumed via SET ROLE
`sdi_admin`	Staff. All three bands	no	assumed via SET ROLE
`sdi_integration`	The GoHighLevel worker. No access to core at all	yes	`demo_int_pw`
`sdi_test_admin`	Test fixtures only. `BYPASSRLS`	yes	`demo_test_pw`

Those passwords come from `sql/99_local_logins.sql`, exist so the demo stack is connectable, and are published in a public repository. They are worth exactly nothing and that file must not be loaded anywhere that matters. `sdi_test_admin` carries `BYPASSRLS` and belongs only in a test database.

4. Architecture



4.1 Four schemas, and the separation is load-bearing

Schema	Holds	Who has USAGE
`core`	Base tables. Properties, people, deals, assignments	No application role. Name resolution fails before any ACL or policy is consulted, so there is no path around the views.
`sec`	Security predicates, definer-rights	All persona roles. They live here because a `security_invoker` view resolves the functions it calls as the caller too, not just the tables.
`api`	Masking views and the write path	All persona roles. The only surface the application reads.
`ghl`	The GoHighLevel bridge	`sdi_integration` only. The web tier cannot enumerate CRM contacts, invoices or transactions.

Brand is a lens, not a property attribute. Both brands read the same `core.property` rows; `core.property_brand` decides publication and price per brand. Adding the second brand is rows in one table, not a schema refactor.

4.2 Tables

Schema	Tables
`core`	`brand`, `person`, `property`, `property_brand`, `property_assignment`, `saved_property`, `pipeline`, `pipeline_stage`, `deal`, `deal_stage_history`
`ghl`	`id_map`, `webhook_event`, `transaction`, `fee_agreement`, `sync_state`, `outbox`, `review_queue`, `reviewable_field`

All ten `core` tables have row-level security enabled and forced; none of the `ghl` tables do, because that schema is reachable only by the integration worker and by nothing else.

Every column of every table is defined in Appendix A, with its type, nullability, default, key role and foreign key target. That appendix is generated from the live database rather than transcribed, so it cannot describe a schema the system does not actually have.

5. The API

5.1 Database views — what the application reads

View	Returns	Granted to
`api.property`	Properties with band 2 masked or released per caller	public, investor, agent, admin

View	Returns	Granted to
`api.property_internal`	Band 3: acquisition cost, source, notes, margin	admin only
`api.my_saved`	The caller's own saved properties	investor, admin
`api.deal`	Deals the caller is party to, with stage and age	investor, agent, admin
`api.deal_history`	Stage transitions for those deals	investor, agent, admin
`api.review_open`	Inbound CRM edits awaiting a decision	admin only

Note that `api.deal` is not granted to `sdi_public` at all. A deal is never public, at any stage.

5.2 Callable functions

Function	Does	Caller
`api.save_property(uuid)`	Saves a property to the caller's list, rejecting anything they cannot see	investor
`api.review_decide(bigint, text)`	Accept or reject a queued CRM edit; accepting writes an allowlist of columns only	admin
`api.security_invariants()`	Returns zero rows, or names what is broken	admin
`ghl.apply_fee_agreement(text)`	Opens the gate for a person, if the document is completed AND paid	integration worker

The `sec.*` predicates (`can_see_address`, `can_see_deal`, `is_assigned`, `actor`, `jitter` and others) are internal. They are the shared source of truth that policies, views and write functions all consult, so a rule cannot drift between the place it is read and the place it is enforced.

5.3 HTTP endpoints

Service	Method and path	Purpose
Web demo	`GET /`	The demo interface
Web demo	`GET /api/view?persona=&brand=`	Everything one persona sees, in one payload
Web demo	`GET /api/probe?persona=`	Attempts a direct base-table read, to show the refusal
Worker	`POST /webhooks/ghl`	Receives a GoHighLevel delivery, verifies its signature
Worker	`GET /healthz`	Queue depth: pending events, bad signatures, outbox backlog, open reviews

The web demo runs on port 3000, the worker on 3001. Neither is authenticated; neither should be exposed to a network until section 8 is addressed.

6. The GoHighLevel Integration

Built against GoHighLevel's official OpenAPI specifications. The full API contract is documented separately in `docs/GHL-Interface-Specification.pdf` (17 pages).

6.1 What the worker does

Component	Responsibility
`ghlClient.js`	HTTP client. Sets the required `Version: 2021-07-28` header once, paces under the 100-request/10-second limit, retries 429 and 5xx, never retries 403
`signature.js`	Verifies webhook signatures against GoHighLevel's published RSA public key
`webhookReceiver.js`	Records deliveries, deduplicating on `webhookId` and rejecting stale ones
`handlers.js`	Dispatches received events
`outbox.js`	Drains outbound writes with retry that cannot duplicate a record
`sync/documents.js`	Polls fee agreement state
`sync/transactions.js`	Nightly reconciliation of payments
`migrate/`	EspoCRM to GoHighLevel load, in ordered passes

6.2 The fee gate has two conditions, not one

GoHighLevel reports document status and payment status *independently*. A document can be signed and unpaid. `ghl.apply_fee_agreement()` is the only thing in the system that opens the gate, and it requires both: completed or accepted, **and** paid. A tag-based unlock gets this wrong.

6.3 Direction decides what happens to an inbound event

`core.property` is the system of record; GoHighLevel is downstream of it for listings. So an inbound record edit means somebody changed a property inside the CRM, against the grain of the architecture. Applying it would let the CRM silently overwrite the authoritative row. Dropping it would lose a real edit. It is neither: it goes to `ghl.review_queue` for a person to decide.

Events that genuinely originate in GoHighLevel — a signed document, a settled payment, a contact created by staff — are applied directly, because for those GoHighLevel is the source. Accepting a queued edit writes only an allowlist of band 1 columns, so a status change can never become a route to rewriting an address or a cost basis.

6.4 One item is unverified

GoHighLevel publishes the webhook public key but does not document the signature algorithm. The code uses PKCS#1 v1.5 with SHA-256, the conventional pairing for that key format. One captured live delivery would settle it, and until then the receiver should not be trusted in production. `worker/tools/capture-webhook.js` **exists to capture one: run it somewhere GoHighLevel can reach, send a test contact through, and it writes the raw bytes and headers untouched.**

7. What Is Tested

Not a coverage percentage. These are the specific claims that are checked, and the attacks that are run against them.

Suite	Checks	Notable
`sql/05_tests.sql`	11	Five attacks: direct base-table read, another investor's saved list, saving an invisible property, a VOLATILE predicate side-channel, and the standing invariants
`sql/07_ghl_tests.sql`	7	A signed-but-unpaid document must not open the gate; replay must not move a signature timestamp
`sql/10_review_tests.sql`	7	A payload smuggling an address and a cost basis alongside a legitimate status change; only the allowlist is applied
`sql/14_pipeline_tests.sql`	9	One agent cannot read another's deal by naming its id; one investor cannot read another's stage history
`worker/ (npm test)`	63	Signature forgery, replay, oversized bodies, rate-limit handling, ambiguous-create duplication, migration resumability

`api.security_invariants()` must always return zero rows. It catches the four changes that quietly dismantle the model: USAGE granted on `core`, an internal column exposed to a non-admin, RLS switched off on a protected table, or a view created without `security_invoker`. Wire it into CI and a nightly check.

8. What Is Not Built

Stated plainly so nothing here is mistaken for finished.

Missing	Consequence
Authentication	No login, no sessions, no passwords for people. The demo selects a persona. This is the gap between the current system and anything user-facing.
The public marketplace UI	Only the demo interface exists. It exercises the model; it is not the product.
Document storage	The signed PDF artifact is not stored
Messaging and unified inbox	Not started
Audit trail on band 2 and 3 reads	Who viewed which address, and when, is not recorded
Co-investment matching	The pipeline exists; the matching engine does not
The external status scraper	Not built. The review queue that would receive its output is.
EspoCRM field mapping	The load ordering and resumability are built and tested. The field-level mapping needs the live EspoCRM schema.
Any deployment	Nothing is hosted. Nothing is internet-facing.

9. Next Steps

Section 8 lists what is missing. This section says what each item needs, in what order, what has to be true before a phase can start, and how each one is tested. Durations are working weeks for one experienced developer and are estimates, not commitments.

9.1 What each item needs, and how long

Ordered as recommended in 9.2. Estimates are working weeks for one experienced developer who already knows this codebase, and cover build plus the tests in 9.4. They exclude design iteration, review latency, and waiting on a third party — the three things that actually move dates.

#	Item	What it is	Needs before starting	Est.
P0	Signature verification and first deployment	Confirm how GoHighLevel signs webhooks, and deploy the receiver that checks it.	A public HTTPS host and a GoHighLevel account. Nothing else in the system.	1w
P1	Authentication and sessions	Real sign-in. A session resolves to a person id and a role, which the web tier assumes for the transaction.	A decision on identity provider: own accounts, or an external one. No database change.	3w
P2	EspoCRM field mapping and rehearsal	Which EspoCRM field becomes which column, and which of the 30+ SDI metrics are inputs rather than derived.	Read access to live EspoCRM or a full export. Load ordering already built and tested.	4w
P3	Audit trail on gated reads	A record of who saw which address, and when.	P1. A decision on where band 2 is released from — PostgreSQL cannot trigger on SELECT.	2w
P4	Public marketplace UI	The real browsing surface: search, filters, property cards, masked map.	P1 for the gated half, P2 for real content, and design direction.	4w
P5	Investor portal and fee flow	Registration, the fee agreement, and the unlock.	P4, and a payment provider connected in GoHighLevel. The CRM side is already built.	3w
P6	Document storage	The signed PDF kept as an artifact, not only as a status flag.	A storage and retention decision. The signed document is a financial record.	2w
P7	Agent portal	An agent's own assignments and conversations, and nothing else.	P1. Per-agent isolation is already enforced and tested at the database level.	2w
P8	Messaging and unified inbox	Email, SMS and WhatsApp threaded against a contact and a property.	A decision on whether GoHighLevel owns the conversation or only relays it. That sets the data model, so settle it first.	3w
P9	External status feed	Nightly check that a listing has not gone pending or sold elsewhere.	A source. A licensed feed is strongly preferable to scraping a portal that forbids it.	2w
P10	Co-investment matching	Pairing two vetted investors on one property.	The matching rules, in writing. The COINVEST pipeline and stages already exist.	3w

#	Item	What it is	Needs before starting	Est.
P11	Hardening and cutover	Production configuration, backups, restore rehearsal, go-live.	Everything above that is in scope for launch.	2w

Total build effort is roughly **31 developer-weeks**. The calendar in 9.2 is 20 weeks because P2, P7 and P9 run alongside other work rather than after it. With one developer and no parallelism the same scope is about 31 weeks; the difference is entirely whether the EspoCRM and operations tracks can proceed independently.

9.2 Recommended sequence

Two things drive this order. **Authentication gates everything user-facing**, so it goes first and almost nothing can be demonstrated to a real user before it lands. And **the audit trail should precede real investor data**, not follow it — the first time anyone asks who saw an address, the answer has to already exist.

P0 is deliberately tiny and first. It deploys nothing but a webhook receiver, which authenticates by signature rather than by session, so it needs no login and exposes no data. It settles the one unverified item in the system and proves the deployment path at the same time.

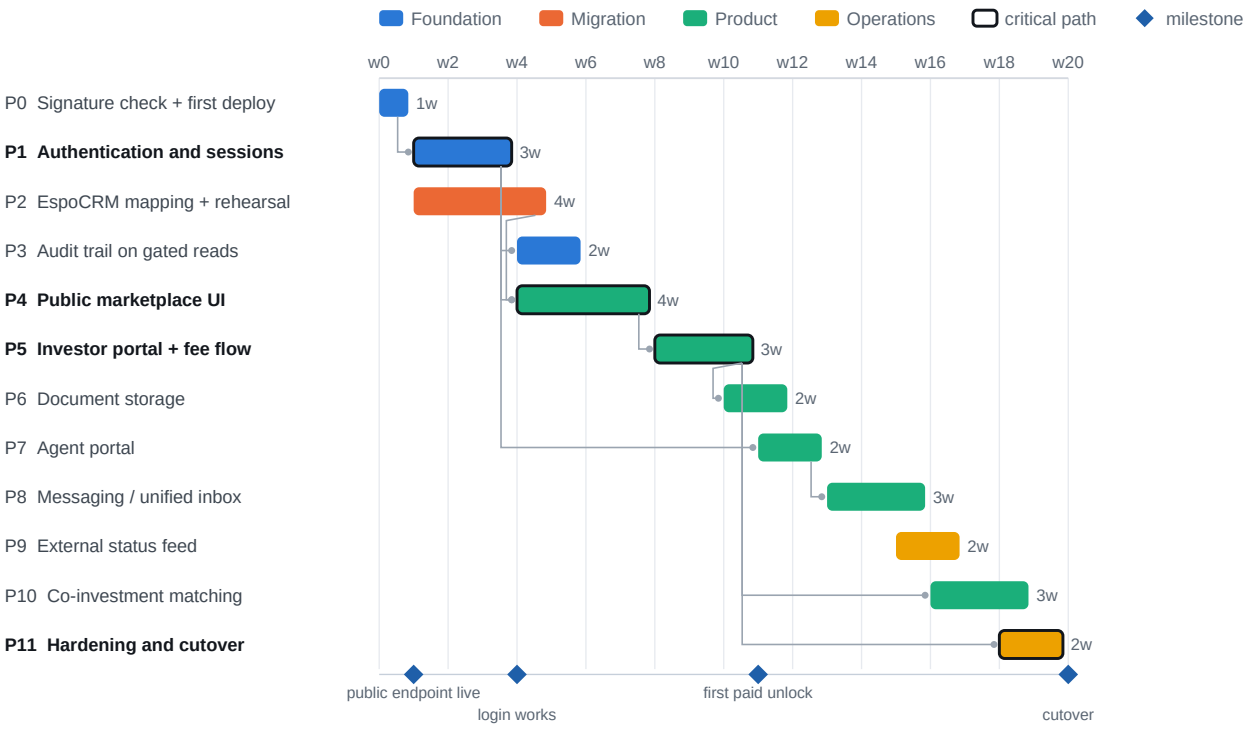


Figure 1. Indicative schedule, one developer. P2 runs alongside P1 because the migration work needs EspoCRM access rather than the new authentication, so the two do not contend.

The critical path is P1 → P4 → P5: authentication, then the browsing surface, then the paid unlock. Everything else can move without moving the launch date. If the schedule has to compress, P8, P9 and P10 are the ones to defer — none of them is required for an investor to find a property, pay, and see the address.

9.3 What must be true before a phase starts

Phase	Entry condition	Done when
P0	A host with a public HTTPS address and a GoHighLevel account	A live delivery verifies against the published key, and the algorithm is confirmed in code and in the spec document
P1	A decision on the identity provider: own accounts, or an external one	A session resolves to a person id and role; every existing SQL walkthrough still passes unchanged
P2	Read access to live EspoCRM, plus the field mapping agreed in writing	A full rehearsal load into a disposable sub-account reconciles with zero shortfall and zero unresolved links

Phase	Entry condition	Done when
P3	P1 complete. A decision on where band 2 is released from	Every band 2 and band 3 read is attributable to a person and a time; the attack tests still pass
P4	P1 complete; design direction; listing content from P2	An anonymous visitor can browse and filter, and cannot obtain an address by any route including the network tab
P5	P4 complete; a payment provider connected in GoHighLevel	An investor signs, pays, and the address unlocks — driven end to end against a GoHighLevel test sub-account
P6	A storage and retention decision	The signed PDF is retrievable and its retention is enforced
P7	P1 complete	An agent sees only their own assignments and conversations, proven by test, not by inspection
P8	The ownership decision in 9.1	A thread is readable against both the contact and the property
P9	A data source	A status change reaches the review queue and no listing changes without a human
P10	Matching rules in writing	Two vetted investors are matched and both are notified
P11	Everything above that is in scope for launch	Cutover rehearsed on staging, with a tested rollback

9.4 How each phase is tested

Every phase adds its own tests and must leave the existing ones green. That second half is the part that usually erodes, so it is stated as a gate rather than a habit.

The standing regression suite

Suite	Checks	Must remain
`sql/05_tests.sql`	11	All five attacks refused
`sql/07_ghl_tests.sql`	7	Signed-but-unpaid still does not open the gate
`sql/10_review_tests.sql`	7	The allowlist still refuses band 2 and band 3 columns
`sql/14_pipeline_tests.sql`	9	No cross-agent or cross-investor read
`worker/ npm test`	63	All passing
`api.security_invariants()`	—	Zero rows, on every build

Run the whole thing with `./run.sh`, which loads the schema, runs all four walkthroughs and the worker suite in one pass. Wire `api.security_invariants()` into CI and into a nightly job: it catches the four changes that quietly dismantle the model, and it catches them whether they came from a migration, a hotfix, or a well-meant grant.

Per-phase development tests

Phase	New tests it must bring
P0	Signature verification against a captured live delivery, added as a fixture so it is checked forever after
P1	Session to role mapping; an expired session; a tampered session; a session for a deactivated person
P2	Reconciliation counts; a resumed load after a forced failure; a link whose endpoints are missing
P3	An audit row exists for every band 2 and band 3 release; the audit log cannot be written by the reader
P4	An anonymous HTTP response contains no restricted field — asserted on the response body, not the rendered page
P5	Signed-and-paid unlocks; signed-and-unpaid does not; a refund after unlock
P6	The stored artifact matches what was signed; retention removes it on schedule
P7	One agent cannot reach another's assignment or conversation by id
P8	A message is attributable to a contact and a property; no cross-tenant leak
P9	A source change raises exactly one review item; a flapping source does not raise many
P10	A match requires two vetted parties; an unvetted party is never matched
P11	A restore from backup produces a working system; rollback returns to the prior version

Two habits worth keeping, both learned building what already exists. Write the test that tries the attack, not only the one that proves the feature — several of the checks in the suite exist because the naive version of a test passed while

proving nothing. And re-run the whole suite, not the file you are working on: two of the bugs found so far only appeared when suites ran together.

9.5 Validation methods to consider

Section 9.4 says what to test. This says *how*, and which technique earns its place where. Not all of these are worth adopting; they are listed with the judgement attached rather than as a checklist to complete.

Method	What it is good for here	Worth it?
Integration tests against a real PostgreSQL	Row-level security, policies and grants cannot be mocked — a mock will happily return rows the real database would refuse. The existing suite already works this way.	Already in use. Keep it; never substitute mocks for the database.
Adversarial tests	A test that tries the attack, not one that proves the feature. Every 'ATTACK' check in the SQL walkthroughs is one, and several exist because the naive version passed while proving nothing.	Essential. One per privilege boundary, minimum.
Property-based testing	Generate random (viewer, property, entitlement) triples and assert the invariant: a viewer without a settled agreement never receives a street address. This explores combinations nobody thought to write down.	High value from P3 onward. The visibility model is an unusually good fit.
Contract testing against the OpenAPI specs	Validate outbound request shapes against GoHighLevel's published specification, so a field rename is caught at build time rather than as a 422 in production.	Worth it once P2 volume is real.
Fault injection	Deliberately fail mid-operation: kill the worker between an API call and its database write, redeliver a webhook, duplicate an outbox row. The resumability claims are only true if they survive this.	Essential before P11. Cheap to run, and the failure it catches is data corruption.
Load testing	The GoHighLevel burst limit is 100 requests per 10 seconds shared across all callers. Confirm the cached read model absorbs concurrent browsing rather than passing it through.	Before P4 goes public. Model realistic concurrency, not a synthetic peak.
Penetration testing	An independent attempt to obtain a street address without paying. The commercial model rests on that being impossible, which makes it the one thing worth an outside opinion.	Before go-live. Scope it explicitly at the gate.
Migration reconciliation	Counts and spot checks between source and destination after every rehearsal, with unresolved links reported rather than dropped.	Already built. Run it on every rehearsal, not just the final one.
User acceptance testing	Walk the seeded personas through real tasks with real staff. Ruth and Marcus exist precisely so the difference is demonstrable to a non-technical observer.	Per phase, from P4.
Accessibility testing	The public marketplace is a consumer surface. Keyboard navigation, contrast and screen reader behaviour on listing cards and filters.	During P4, not after. Retrofitting is far more expensive.
Data quality checks	Standing assertions on the ledger: no negative amounts, no refund exceeding its charge, no transaction pointing at a missing invoice. Some are already constraints.	Promote to constraints where possible — a constraint cannot be forgotten.

Method	What it is good for here	Worth it?
Static analysis and dependency audit	Lint, type checking, and a scan of the dependency tree. This project has one runtime dependency, which keeps this cheap.	In CI from now. The cost is near zero while the tree is small.

Three rules to attach to whichever of these you adopt

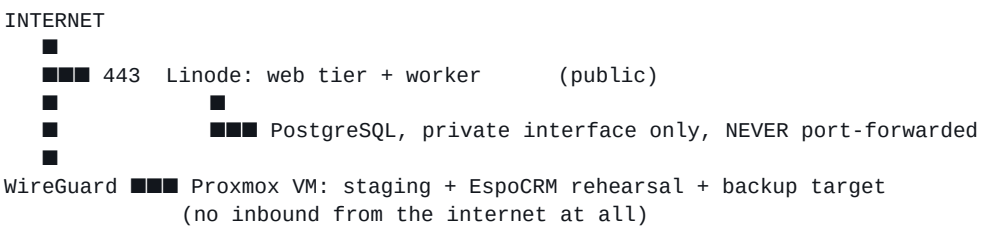
- **Run the whole suite, not the file you are working on.** Two of the bugs found while building this only appeared when suites ran together — one because two test files shared a database and raced.
- **A test that cannot fail is not a test.** Two checks here originally passed for the wrong reason: a permission error fired before the code under test was ever reached. Make each new test fail once, deliberately, before trusting it.
- **Wire `api.security_invariants()` into CI and a nightly job.** It is the only check that catches a policy quietly dismantled by a later migration, and it costs one query.

9.6 Deployment

Both available options are suitable, for different jobs. The recommendation is to use both rather than choose.

Environment	Role	Why
Proxmox VM (local)	Development and staging. The full stack, no public exposure.	The EspoCRM rehearsal belongs here: real client data never leaves the network, and a rehearsal against a snapshot is repeatable in a way a live read is not. A VM rather than a container — PostgreSQL wants a stable filesystem and its own kernel-level tuning.
Linode (public)	Production. Web tier and worker, publicly reachable.	P0 needs a public HTTPS endpoint for GoHighLevel to deliver to, and nothing else does until P4. Start with the smallest instance that runs the worker.

Topology



Four rules for the production environment, each of which is cheap now and expensive to retrofit:

- **PostgreSQL is never exposed to the internet.** Bind it to the private interface. The proxy this project was built behind refuses raw database ports for exactly this reason.
- `sdi_test_admin` **must not exist in production.** It carries BYPASSRLS and exists only for test fixtures. `worker/test/bootstrap.sql` is not a deployment script.
- `sql/99_local_logins.sql` **must not be loaded in production.** Its passwords are published in a public repository. Production roles get credentials from the deployment, not from the repository.
- **The GoHighLevel token lives in the environment**, never in a file and never in browser-reachable code. It is scoped to an entire sub-account.

Backups are the one thing to set up before there is anything worth backing up, because that is the only time it is easy. Nightly `pg_dump` to the Proxmox side over WireGuard, and a restore rehearsed at least once — an untested backup is a belief, not a backup.

10. Where Things Are

Path	Contents
`sql/01–04`	Schema, RLS policies, masking views, demo data
`sql/05, 07, 10, 14`	The four walkthroughs. Each is readable top to bottom as an argument
`sql/06, 08, 09`	GoHighLevel bridge, review queue, review actions
`sql/11–13`	Deals, stage history, pipeline policies and seed
`sql/99_local_logins.sql`	Demo passwords. Local development only
`web/`	The demo: `server.js` and one HTML page
`worker/src/`	The GoHighLevel integration worker
`worker/test/`	63 tests
`worker/tools/`	The webhook capture tool
`docs/`	This document and the GoHighLevel interface specification, with their generators
`README.md`	The same ground in more technical detail, including the reasoning behind each design decision

There is no INSTALL file; section 2 of this document and the README's opening section cover installation. Both PDFs in `docs/` *are generated by committed scripts rather than written by hand, so they can be regenerated as the system changes and cannot quietly drift out of date.*

Appendix A. Table Definitions

Every column of every base table, read from the live database by `docs/extract_schema.py` and stored in `docs/schema-snapshot.json`. Regenerate the snapshot after any schema change and rebuild this document; the appendix cannot then describe a schema the system does not have.

Marker	Meaning
PK	Part of the primary key
FK	Foreign key; the referenced table is named in the same row
UQ	Covered by a unique constraint
CK	Covered by a check constraint
not null	The column is `NOT NULL`

Schema `core`

`core.brand`

4 columns · row-level security enforced

Column	Type	Null	Key	Default / references
`brand_code`	`text`	not null	PK	
`display_name`	`text`	not null		
`service_tier`	`text`	not null	CK	
`platform_fee`	`numeric(10,2)`	not null		

`core.deal`

13 columns · row-level security enforced

Column	Type	Null	Key	Default / references
`deal_id`	`uuid`	not null	PK	gen_random_uuid()
`external_ref`	`text`		UQ	
`property_id`	`uuid`	not null	FK	→ core.property
`investor_id`	`uuid`		FK	→ core.person
`agent_id`	`uuid`		FK	→ core.person
`pipeline_code`	`text`	not null	FK	→ core.pipeline_stage
`stage_code`	`text`	not null	FK	→ core.pipeline_stage
`amount`	`numeric(12,2)`			
`opened_at`	`timestamptz`	not null	CK	now()
`closed_at`	`timestamptz`		CK	
`lost_reason`	`text`			
`created_at`	`timestamptz`	not null		now()

Column	Type	Null	Key	Default / references
updated_at	timestamptz	not null		now()

core.deal_stage_history

7 columns · row-level security enforced · Append-only, written by trigger. The application cannot forget to log a transition, and cannot rewrite one after the fact.

Column	Type	Null	Key	Default / references
id	bigint	not null	PK	auto-increment
deal_id	uuid	not null	FK	→ core.deal
from_stage	text			
to_stage	text	not null		
changed_at	timestamptz	not null		now()
changed_by	uuid			
seconds_in_from	bigint			

core.person

7 columns · row-level security enforced

Column	Type	Null	Key	Default / references
person_id	uuid	not null	PK	gen_random_uuid()
role	actor_role	not null		
full_name	text	not null		
email	text	not null	UQ	
fee_agreement_signed_at	timestamptz			
home_brand	text		FK	→ core.brand
active	boolean	not null		true

core.pipeline

4 columns · row-level security enforced

Column	Type	Null	Key	Default / references
pipeline_code	text	not null	PK	
display_name	text	not null		
brand_code	text		FK	→ core.brand
active	boolean	not null		true

core.pipeline_stage

6 columns · row-level security enforced

Column	Type	Null	Key	Default / references
pipeline_code	text	not null	FK PK UQ	→ core.pipeline
stage_code	text	not null	PK	
display_name	text	not null		
position	integer	not null	UQ	
is_won	boolean	not null	CK	false
is_lost	boolean	not null	CK	false

core.property

27 columns · row-level security enforced

Column	Type	Null	Key	Default / references
property_id	uuid	not null	PK	gen_random_uuid()
listing_ref	text	not null	UQ	
status	text	not null	CK	
city	text	not null		
state	char(2)	not null		
zip	text	not null		
property_type	text	not null		
beds	smallint			
baths	numeric(3,1)			
sqft	integer			
year_built	smallint			
list_price	numeric(12,2)	not null		
gross_rent_annual	numeric(12,2)	not null		
opex_annual	numeric(12,2)	not null		
hoa_annual	numeric(12,2)	not null		0
noi_annual	numeric(12,2)			
cap_rate	numeric(6,4)			
street_address	text	not null		
unit	text			
lat	numeric(9,6)	not null		
lng	numeric(9,6)	not null		
parcel_number	text			
seller_disclosure	text			
acquisition_cost	numeric(12,2)			

Column	Type	Null	Key	Default / references
source_channel	text			
internal_notes	text			
created_at	timestamptz	not null		now()

core.property_assignment

3 columns · row-level security enforced

Column	Type	Null	Key	Default / references
property_id	uuid	not null	FK PK	→ core.property
person_id	uuid	not null	FK PK	→ core.person
assign_role	text	not null	CK PK	

core.property_brand

4 columns · row-level security enforced

Column	Type	Null	Key	Default / references
property_id	uuid	not null	FK PK	→ core.property
brand_code	text	not null	FK PK	→ core.brand
published	boolean	not null		false
brand_price	numeric(12,2)			

core.saved_property

3 columns · row-level security enforced

Column	Type	Null	Key	Default / references
person_id	uuid	not null	FK PK	→ core.person
property_id	uuid	not null	FK PK	→ core.property
saved_at	timestamptz	not null		now()

Schema ghl

ghl.fee_agreement

10 columns · no row-level security

Column	Type	Null	Key	Default / references
document_id	text	not null	PK	
location_id	text	not null		
person_id	uuid		FK	→ core.person
ghl_contact_id	text			
status	text	not null		
payment_status	text	not null		

Column	Type	Null	Key	Default / references
grand_total	numeric(12,2)			
ghl_updated_at	timestamptz	not null		
observed_at	timestamptz	not null		now()
raw	jsonb	not null		

ghl.id_map

6 columns · no row-level security

Column	Type	Null	Key	Default / references
entity_type	text	not null	PK	
local_id	uuid	not null	PK	
ghl_id	text	not null	UQ	
ghl_object	object_kind	not null		
location_id	text	not null	PK UQ	
synced_at	timestamptz	not null		now()

ghl.outbox

13 columns · no row-level security

Column	Type	Null	Key	Default / references
id	bigint	not null	PK	auto-increment
idempotency_key	text	not null	UQ	
operation	text	not null		
entity_type	text			
local_id	uuid			
payload	jsonb	not null		
state	outbox_state	not null		'pending'
attempts	integer	not null		0
next_attempt_at	timestamptz	not null		now()
last_error	text			
ghl_id	text			
created_at	timestamptz	not null		now()
sent_at	timestamptz			

ghl.review_queue

12 columns · no row-level security

Column	Type	Null	Key	Default / references
id	bigint	not null	PK	auto-increment

Column	Type	Null	Key	Default / references
source	text	not null		
event_type	text	not null		
ghl_object	text			
ghl_id	text			
local_id	uuid			
summary	text	not null		
proposed	jsonb	not null		
state	review_state	not null		'open'
raised_at	timestamptz	not null		now()
decided_at	timestamptz			
decided_by	uuid			

ghl.reviewable_field

1 columns · no row-level security · Allowlist. An accepted CRM edit can write these and nothing else. Deliberately excludes every band 2 and band 3 column: a status change must not be a route to rewriting an address or a cost basis.

Column	Type	Null	Key	Default / references
column_name	text	not null	PK	

ghl.sync_state

7 columns · no row-level security

Column	Type	Null	Key	Default / references
resource	text	not null	PK	
location_id	text	not null		
cursor_at	timestamptz			
last_run_at	timestamptz			
last_ok_at	timestamptz			
last_error	text			
records_seen	bigint	not null		0

ghl.transaction

17 columns · no row-level security

Column	Type	Null	Key	Default / references
ghl_id	text	not null	PK	
location_id	text	not null		
contact_id	text			
invoice_id	text			

Column	Type	Null	Key	Default / references
subscription_id	text			
amount	numeric(12,2)	not null	CK	
currency	char(3)	not null		
amount_refunded	numeric(12,2)	not null	CK	0
status	text	not null		
live_mode	boolean	not null		
payment_provider	text			
entity_type	text			
entity_id	text			
ghl_created_at	timestamptz	not null		
ghl_updated_at	timestamptz	not null		
synced_at	timestamptz	not null		now()
raw	jsonb	not null		

ghl.webhook_event

9 columns · no row-level security

Column	Type	Null	Key	Default / references
webhook_id	text	not null	PK	
event_type	text	not null		
occurred_at	timestamptz	not null		
received_at	timestamptz	not null		now()
processed_at	timestamptz			
attempts	integer	not null		0
last_error	text			
signature_ok	boolean	not null		
payload	jsonb	not null		